Atividade — Code Smells e Refatoração

Disciplina: Engenharia de Software
Professora: Patricia Rucker de Bassi
Tema: Tornar o código mais compreensível / evitar code smells
Integrante: Lucas Muller Scuzziato
Data: 26/10/2026
Material de apoio: slides da aula 4.2 — Manutenção de software
(material/slides/Engenharia de Software - aula 4.2 - manutencao de software.pdf)

Consulta a LLM: a pesquisa e a organização dos exemplos foram feitas com ChatGPT (versão gratuita / GPT básico).

Introdução

Code smells são indícios de que o código pode estar difícil de entender, alterar ou testar. Não são bugs por si só, mas aumentam o custo de manutenção. Abaixo, três smells clássicos, cada um com benefícios da refatoração, como refatorar e um exemplo curto em Python (versão com smell e versão refatorada).

---

1. Long Method (Método Longo)

1.1 Nome do Code Smell

Long Method — um método/função faz muitas coisas de uma vez e fica longo demais para ser lido de uma só vez.

1.2 Benefícios de refatorar

- Fica mais fácil entender a intenção de cada trecho.
- Facilita testes unitários (funções menores e focadas).
- Reduz risco de misturar responsabilidades e esconder bugs.
- Torna futuras alterações mais localizadas.

1.3 Como refatorar

1. Identificar blocos com responsabilidade clara (validação, cálculo, formatação, etc.).
2. Extrair cada bloco para uma função com nome que descreva o quê ela faz (Extract Method).
3. Deixar a função original como orquestração curta que chama as partes.

1.4 Exemplo em Python

Com smell (Long Method):

```
def processar_pedido(itens, cupom):
    # valida, calcula, aplica desconto e monta o texto — tudo numa função só
    total = 0
    for item in itens:
        if item["qtd"] <= 0 or item["preco"] < 0:
            raise ValueError("item inválido")
        total += item["qtd"] * item["preco"]

    if cupom == "DESC10":
        total = total * 0.9
    elif cupom == "DESC20":
```

```
        total = total * 0.8

    linhas = []
    for item in itens:
        linhas.append(f"{item['nome']}: {item['qtd']} x {item['preco']:.2f}")
    linhas.append(f"Total: R$ {total:.2f}")
    return "\n".join(linhas)
```

Refatorado:

```
def validar_itens(itens):
    for item in itens:
        if item["qtd"] <= 0 or item["preco"] < 0:
            raise ValueError("item inválido")


def calcular_subtotal(itens):
    return sum(item["qtd"] * item["preco"] for item in itens)


def aplicar_desconto(total, cupom):
    descontos = {"DESC10": 0.9, "DESC20": 0.8}
    return total * descontos.get(cupom, 1.0)


def formatar_recibo(itens, total):
    linhas = [
        f"{item['nome']}: {item['qtd']} x {item['preco']:.2f}"
        for item in itens
    ]
    linhas.append(f"Total: R$ {total:.2f}")
    return "\n".join(linhas)


def processar_pedido(itens, cupom):
    validar_itens(itens)
    total = aplicar_desconto(calcular_subtotal(itens), cupom)
    return formatar_recibo(itens, total)
```

---

2. Duplicate Code (Código Duplicado)

2.1 Nome do Code Smell

Duplicate Code — o mesmo (ou quase o mesmo) trecho de lógica aparece em mais de um lugar.

2.2 Benefícios de refatorar

- Correção de bug ou mudança de regra é feita em um único ponto.
- Menos chance de inconsistência entre cópias “quase iguais”.
- Código mais curto e mais fácil de revisar.
- Facilita reuso consciente.

2.3 Como refatorar

1. Localizar trechos iguais ou muito parecidos.
2. Extrair a lógica comum para uma função/módulo compartilhado (Extract Method / Pull Up Method).
3. Substituir as cópias por chamadas a essa abstração.
4. Se houver pequenas variações, parametrizar em vez de copiar.

2.4 Exemplo em Python

Com smell (Duplicate Code):

```
def media_turma_a(notas):
    if not notas:
        return 0.0
    return sum(notas) / len(notas)


def media_turma_b(notas):
    if not notas:
        return 0.0
    return sum(notas) / len(notas)


def media_turma_c(notas):
    if not notas:
        return 0.0
    return sum(notas) / len(notas)
```

Refatorado:

```
def media(notas):
    if not notas:
        return 0.0
    return sum(notas) / len(notas)


# Uso: media(notas_a), media(notas_b), media(notas_c)
```

---

3. Magic Numbers (Números Mágicos)

3.1 Nome do Code Smell

Magic Numbers — literais numéricos (ou strings “soltas”) aparecem no código sem nome que explique o significado.

3.2 Benefícios de refatorar

- O leitor entende por que aquele valor existe.
- Mudança de regra de negócio (ex.: limite, taxa) fica centralizada.
- Menos erros ao alterar o mesmo número em vários pontos.
- Código mais autodocumentado.

3.3 Como refatorar

1. Dar nome significativo a cada literal importante (constante ou variável bem nomeada).
2. Agrupar constantes relacionadas no topo do módulo ou em um lugar único de configuração.
3. Evitar repetir o mesmo literal espalhado pelo código.

3.4 Exemplo em Python

Com smell (Magic Numbers):

```
def calcular_frete(peso_kg, distancia_km):
    # O que significam 5, 2.5, 0.15 e 12?
    if peso_kg > 5:
        return 12 + distancia_km * 0.15
    return 2.5 + distancia_km * 0.15
```

Refatorado:

```
PESO_LIMITE_KG = 5
```

```
TAXA_POR_KM = 0.15
TAXA_BASE_LEVE = 2.5
TAXA_BASE_PESADO = 12.0


def calcular_frete(peso_kg, distancia_km):
    taxa_base = TAXA_BASE_PESADO if peso_kg > PESO_LIMITE_KG else TAXA_BASE_LEVE
    return taxa_base + distancia_km * TAXA_POR_KM
```

Conclusão

Os três smells tratados — Long Method, Duplicate Code e Magic Numbers — atrapalham a compreensão e a evolução do código. Refatorar com extrair métodos, eliminar duplicação e nomear constantes deixa o software mais legível, testável e barato de manter, alinhado às boas práticas de manutenção de software.

Entrega individual — Lucas Muller Scuzziato · consulta realizada com ChatGPT (gratuito / GPT básico).